

Labwork 5 - Shared memory optimization

Do Minh Quang - 2540095
University of Science and Technology of Hanoi

1 Shared memory optimization for Gaussian filter

In this labwork, we replace the grayscale kernel with a Gaussian blur kernel. And we also learn how to use shared memory to improve the performance. The experiment uses 718×718 RGB image.



Figure 1: Input image.

The blurry output image is generated by applying a 7×7 Gaussian blur filter to the input image. The global memory implementation directly accesses the filter values from global memory during the convolution process. Since the same filter elements are repeatedly accessed by multiple threads, this results in redundant global memory reads.

In the shared memory implementation, the filter is first loaded into shared memory by the threads in each block. After synchronization using `cuda.syncthreads()`, all threads in the block can reuse the filter values from the faster shared memory instead of accessing global memory repeatedly.

2 Results

For the qualitative results, the image is blurred after applying the Gaussian filter. The CPU and GPU implementations produce the same blur quality when applying the same Gaussian filter, as the difference between them lies in computational performance rather than output quality.



Figure 2: Comparison of Gaussian blur implementations. From left to right: a) original image, b) CPU implementation, c) GPU global memory implementation, d) GPU shared memory implementation.

For the quantitative performance comparison, all implementations are executed 10 times, and the average execution time and standard deviation are recorded. A warm-up execution is performed before measurement to reduce the impact of initial CUDA overhead. CPU computation took an average of 68.1927 seconds, while the GPU global memory implementation took 0.0070 seconds, and the GPU shared memory implementation took 0.0061 seconds. Both GPU implementations are significantly faster than the CPU, achieving speedups of approximately $9742\times$ and $11179\times$ for the global and shared memory versions, respectively. The shared memory implementation is approximately 12.9% faster than the global memory version.

2.1 Conclusion

GPU parallelization substantially reduces execution time compared to the CPU. Using shared memory further improves performance by enabling efficient reuse of filter values within each thread block.